

1. System Architecture

For the system Architecture, it is built upon a modular pipeline, this is to maximize throughput and maintain in task ownerships where the pipeline separates responsibilities to do such tasks.

Several key factors in the system are composed of:

- **Main rs:** The entry point in rust, this is where the execution environment is initialized. Some of the tasks include Initialization where it handles command line arguments and variables. Another task that it handles is lifecycle management, handles the primary data structures and the teardown of threads to prevent leaks.
- **generator.rs:** For performance this acts as the source in a queuing system, using fixed seed it stimulates the arrival of work this is done by creating synthetic tasks that happen over time. The fixed seed acts as the random number generator that ensures the handling of tasks across multiple test runs.
- **dispatcher.rs:** The mastermind of the system that manages the control plane, this is the handler of the ready queue and lets available workers know about forwarded tasks. It maintains a FIFO queue that contains the tasks that are either “ready” or “running”.
- **work.rs:** Manages a pool of eight workers threads, the system implements resource throttling that prevents excessive context switching that occurs after each spawn of a new thread for every individual task.

- **metrics.rs and monitor.rs:** These modules handle two things respectively, the tracking of timing statistics and the reporting of the final results. The metrics captures raw data while the monitor aggregates the raw data from metrics to do the necessary calculations in order to accomplish formatting of human readable

The structure is responsible for tracking a task from its original creation in the generator thread, from it's waiting time in the dispatcher queue then to its final execution.

2. Task Model and Scheduling Design

With the simulation each task carries essential metadata that is required to make scheduling decisions: that includes a, unique **ID**, an **arrival time**, the **kind** (CPU or I/O), and it's **duration**. To ensure that the workload pattern remains identical in different runs we use a fixed seed that allows a fair comparison between the scheduling strategies. As mentioned before the dispatcher implements a First-in, First-Out queue. This queue is a valid way to achieve the goal of handling tasks, because it acts as a buffer when tasks arrivals outpace the worker pool. However, despite the significance of it there is a trade off to using it, which is the fact that FIFO is head-of-line blocking. This means that when there is a CPU heavy task in queue it can delay some tasks that are behind it. So, to counter, an optimized scheduling policy was added based on the task duration in order to minimize the wait time. This policy is the Shortest Job First (**SJF**) so the shortest tasks will be executed first.

3. Synchronization Strategy

Utilizing Rust core concurrency primitives, we can ensure thread safety and prevent data races using the following strategies:

- **Channels (`std::sync::mpsc`):** In order to handle the task handoff between the generator, dispatcher and workers we use channels. This allows ownership of a task to ensure that it is transferred clearly. Due to this, the sender can no longer modify the task once it is sent.
- **Shared State (`Arc<Mutex<Metrics>>`):** Since multiple workers have to update any statistics as the tasks finish, the strategy of using an Arc (Atomic reference counter) is to ensure thread safe shared ownership. Also, a mutex is wrapped around the metrics to make sure the updates are atomic. This is done to prevent race conditions when handling task statistics.

4. Deterministic Shutdown

To prevent the simulation from dropping telemetry data, the system will follow a termination sequence that is structured:

- **Completion Signaling:** Once all 1,000 tasks are produced by the generator, a signal is issued to the dispatcher to indicate that no further work is next.
- **Thread Joining:** As stated, before `main.rs` remains vital, `main.rs` manages `JoinHandles` for each thread. By calling `join()` the system guarantees that each task is completed by every worker and gives a clean report of all the metrics to

the reporting module. This process allows the program to only exit after all activity is done.

5. Experiment results: FIFO vs. Optimized

Performance Metrics Table

Metric	Experiment A: Balanced (50/50)	Experiment B: Stressed (80/20)
Total Tasks	1,000	1,000
Makespan (Total Time)	25.31s	25.45s
Average Wait Time	2.05s	1.75s
Max Wait Time	4.08s	3.47s
Peak Queue Length	1	1

Analysis of Comparison

The experimental data demonstrates that the system maintained high efficiency and stability across both testing scenarios, with the total make span only varying by 0.14 seconds. In Experiment A, the balanced workload resulted in an average wait time of 2.05s, while Experiment B surprisingly showed a decrease in both average (1.75s) and maximum (3.47s) wait times despite the higher CPU stress. Crucially, the Peak Queue Length remained at 1 for both trials, proving that the dispatcher effectively prevented bottlenecks and handled task arrivals in real-time without saturating the worker pool. These results suggest that while the FIFO policy is fair, the system’s high worker utilization and low turnaround times ensure reliable performance regardless of whether the workload is balanced or CPU-heavy.